Linux Commands

1. basename -a

Prints the final component of each supplied path. The -a option allows multiple paths.

Command:

```
basename -a /home/Srini/file1.txt /etc/passwd
```

Example output:

```
file1.txt
passwd
```

Explanation: The directory portion of each path is removed. This command does not modify files.

2. cat fileA.txt

Displays the contents of fileA.txt.

Command:

```
cat fileA.txt
```

Example output:

```
apple
banana
mango
```

Explanation: cat is useful for reading short text files in the terminal.

3. cat fileB.txt

Displays the contents of fileB.txt.

Command:

```
cat fileB.txt
```

Example output:

```
apple
grapes
mango
```

Explanation: The output depends on the file contents.

4. comm fileA.txt fileB.txt

Compares two sorted files and displays three tab-separated columns.

Command:

```
comm fileA.txt fileB.txt
```

Example output:

```
                apple

banana
        grapes
                mango
```

Explanation: Column 1 contains lines only in the first file; column 2 contains lines only in the second file; column 3 contains lines common to both. Inputs should be sorted. In the screenshot, apple and mango are common, banana is only in fileA.txt, and grapes is only in fileB.txt.

5. join fileA.txt fileB.txt

Combines lines from two sorted files when their first fields match.

Command:

```
join fileA.txt fileB.txt
```

Example output:

```
apple
mango
```

Explanation: join matches the first whitespace-separated field by default. The screenshot shows apple and mango in both files, so those matches are printed.

6. cat file1.txt

Prints the contents of file1.txt.

Command:

```
cat file1.txt
```

Example output:

```
Hi
Hello
How
are
you?    are you     ok?     i   am  ok
```

Explanation: The final line contains spaces or tab spacing between words. cat displays the text but does not distinguish tabs from spaces visually.

7. tac file1.txt

Prints a file with its lines in reverse order.

Command:

```
tac file1.txt
```

Example output:

```
you?    are you     ok?     i   am  ok
are
How
Hello
Hi
```

Explanation: tac reverses line order, not the characters within each line.

8. shuf names.txt

Prints the lines of a file in random order.

Command:

```
shuf names.txt
```

Example output:

```
Example screenshot output:
what
I am currently doing my work...
you doing?
are
```

Explanation: The order can change each time. It does not modify names.txt.

9. timeout 5s sleep 10

Runs a command with a five-second time limit.

Command:

```
timeout 5s sleep 10
```

Example output:

```
Normally no output is printed.
```

Explanation: sleep 10 waits for ten seconds, but timeout stops it after five seconds. The exit status is normally 124 when the time limit is reached.

10. seq 5 15

Prints integers from 5 through 15.

Command:

```
seq 5 15
```

Example output:

```
5
6
7
8
9
10
11
12
13
14
15
```

Explanation: Each number is printed on a separate line.

11. printf '%s\n' *.txt

Prints each filename matching *.txt on a separate line.

Command:

```
printf '%s\n' *.txt
```

Example output:

```
file1.txt
file2.txt
fileA.txt
fileB.txt
names.txt
output.txt
```

Explanation: The shell expands *.txt into matching filenames before printf runs.

12. split -b 10 file1.txt byte_

Splits file1.txt into chunks containing up to ten bytes each.

Command:

```
split -b 10 file1.txt byte_
ls -l byte_*
```

Example output:

```
byte_aa
byte_ab
byte_ac
byte_ad
byte_ae
```

Explanation: -b 10 sets the chunk size to 10 bytes. The final chunk may be smaller. The screenshot shows four 10-byte chunks and one 6-byte chunk.

13. ls -l byte_*

Lists detailed information for files whose names start with byte_.

Command:

```
ls -l byte_*
```

Example output:

```
Example files:
byte_aa
byte_ab
byte_ac
byte_ad
byte_ae
```

Explanation: -l displays permissions, link count, owner, group, size, modification time, and filename. The wildcard * matches the suffix.

14. factor 120

Displays the prime factors of 120.

Command:

```
factor 120
```

Example output:

```
120: 2 2 2 3 5
```

Explanation: 120 = 2 × 2 × 2 × 3 × 5.

15. factor 130

Displays the prime factors of 130.

Command:

```
factor 130
```

Example output:

```
130: 2 5 13
```

Explanation: 130 = 2 × 5 × 13.

16. factor 24

Displays the prime factors of 24.

Command:

```
factor 24
```

Example output:

```
24: 2 2 2 3
```

Explanation: 24 = 2 × 2 × 2 × 3.

Notes

comm and join work best with sorted input files.
shuf changes the display order randomly but leaves the source file unchanged.
split creates output files and does not change the original file.
factor prints the prime factorization of integers.